Détection d'équipements de protection individuelle en temps réel

Rapport technique — Module Deep Learning Avancé (Axe 3, YOLO)

Omar Ghannem
Cycle ingénieur IA & Data Science, 3e année — TEK-UP
Enseignant : Haythem Ghazouani

November 11, 2026

Résumé

Nous présentons un système complet de détection de personnes et d'équipements de protection individuelle (EPI) en milieu de travail, de l'entraînement au déploiement web. Des modèles YOLO sont entraînés sur le dataset public SH17 (8 099 images, 17 classes), suivis avec MLflow, exportés en ONNX et servis par une API FastAPI consommée par une interface React avec webcam. Une règle de conformité déclenche une alerte lorsqu'une tête est détectée sans casque. La chaîne est reproductible (DVC, MLflow, Docker Compose) et testée par GitHub Actions. Le meilleur modèle (YOLOv8n à 960 px) atteint 56,9 % de mAP@50 et 35,0 % de mAP@[.5:.95] sur le jeu de test, proche des 58,0 % publiés, avec 20 epochs seulement ; l'export ONNX divise la latence CPU par 1,5.

1 Problématique et objectifs

Le contrôle du port des EPI est aujourd'hui surtout manuel. Problématique : *comment détecter automatiquement et en temps réel, via une application web, les personnes et leurs équipements de protection, et alerter en cas de non-conformité ?*

Objectifs mesurables : (1) baseline YOLO nano proche des 58,0 % mAP@50 publiés pour YOLOv8n ; (2) comparaison d'au moins deux architectures sur mAP@[.5:.95] et latence ; (3) effet de la résolution d'entrée (640 contre 960) ; (4) export ONNX et gain de latence mesuré ; (5) API et frontend avec webcam et alerte.

2 Données

2.1 Le dataset SH17

SH17 [1] contient 8 099 images issues de Pexels, 75 994 instances et 17 classes, au format YOLO, sous licence CC BY-NC-SA 4.0 (usage académique). L'ordre des identifiants est celui du fichier `sh17.yaml` (alphabétique), différent du README ; nous l'avons vérifié visuellement.

2.2 Exploration et qualité

- Aucune image corrompue, aucune annotation invalide, aucun doublon, aucune fuite train/val.
- Fort déséquilibre : ratio max/min de 118,3 (`hands` 15 850 contre `face-guard` 134) ; les EPI clés sont rares (`helmet` 927, `safety-vest` 530).
- Petits objets : 35,7 % des boîtes font moins de 32 px à 640 px d'entrée (oreilles 92 %, pieds 72 %).
- Densité moyenne de 9,4 objets par image ; résolutions de 1335×1002 à 8192×8192.
- Contenu : surtout des bureaux et ateliers, peu de chantiers.

2.3 Protocole de découpage

Split	Origine	Images
train	90 % du train officiel	5 831
val	10 % du train officiel (seed 42)	648
test	val officiel (comparable au benchmark)	1 620

Le découpage est fait une seule fois, avant toute augmentation, par le stage DVC `prepare` qui réduit aussi les images à 1 280 px de grand côté (les labels normalisés restent valides). Le choix du meilleur modèle se fait sur la validation ; le test ne sert qu'au rapport final.

3 Modélisation

Nous utilisons la famille YOLO (Ultralytics) en transfer learning depuis COCO : un réseau de détection en une passe, adapté au temps réel. Trois runs, même seed et même budget :

1. **baseline** YOLOv8n, 640 px, 30 epochs (même modèle que le benchmark publié) ;
2. **YOLO26n**, 640 px, 30 epochs : architecture récente, sortie sans NMS ;
3. YOLOv8n à **960 px**, 20 epochs : effet de la résolution sur les petits objets.

Augmentations Ultralytics par défaut (mosaïque, HSV, flips, translation/échelle), early stopping (patience 10), sélection sur le mAP@[.5:.95] de validation.

4 Résultats

Run	val mAP50-95 (%)	test mAP50 (%)	test mAP50-95 (%)	AP50 helmet (%)	GPU (ms)
baseline-yolov8n-640	32,0	52,7	32,4	60,7	3,1
yolo26n-640	32,3	53,3	33,2	62,3	3,1
yolov8n-960	**33,7**	**56,9**	**35,0**	**68,5**	7,5
Référence YOLOv8n [1]	–	58,0	36,6	–	–

Tableau 1: Comparaison des modèles (source : `reports/comparison.csv`, MLflow).

Lecture du tableau. Le choix se fait sur la validation : `yolov8n-960` est le meilleur (33,7 % de mAP@[.5:.95]) et le test le confirme. À même résolution, YOLO26n fait un peu mieux que YOLOv8n (+0,6 point de mAP@50, +0,8 point de mAP@[.5:.95]) pour la même latence GPU. Le passage de 640 à 960 px apporte le gain le plus net (+4,2 points de mAP@50, +7,8 points d'AP@50 sur `helmet`), au prix d'une latence GPU 2,5 fois plus élevée. À 640 px, notre baseline (52,7 %) reste sous les 58,0 % publiés : nous n'avons fait que 30 epochs.

Analyse par taille d'objet. Rappel à IoU 0,5 du meilleur modèle sur le test (taille vue par le réseau, racine de l'aire de la boîte) :

Taille	Objets	Détectés	Rappel
<16 px	1 472	337	22,9 %
16–32 px	2 103	1 324	63,0 %
32–96 px	5 518	4 354	78,9 %
>96 px	6 265	5 501	87,8 %

Les très petits objets sont la principale source d'erreurs : moins d'un sur quatre est retrouvé, contre près de 9 sur 10 pour les grands. C'est ce qui explique le gain apporté par la résolution 960 px.

Analyse par classe. Les EPI clés restent les plus difficiles : `helmet` atteint 68,5 % d'AP@50 mais `safety-vest` seulement 39,7 % (530 instances, la classe EPI la plus rare après `face-guard` et `medical-suit`). La règle d'alerte reposant sur `helmet`, un casque manqué produit une fausse alerte.

5 Export ONNX et latence

Le meilleur modèle (`yolov8n-960`) est exporté en ONNX (opset 17, entrée fixe 960×960). Sur le CPU de Kaggle, à entrée identique : PyTorch 216,9 ms contre ONNX Runtime 141,6 ms (médiane sur 50 passages après 10 d'échauffement), soit un gain de 1,53×. L'écart de mAP@50 entre ONNX et PyTorch sur le test étant inférieur à 2 points, la version 2 de `sh17-ppe-detector` est passée de l'alias `staging` à `production` dans le MLflow Model Registry (la version 1, YOLO26n, issue du premier export, reste disponible). Sur un ordinateur portable, l'API mesure environ 190 ms d'inférence à 960 px, soit environ 4 images par seconde en webcam ; le modèle à 640 px serait environ 2 fois plus rapide.

6 Architecture logicielle et MLOps

- **DVC** : `dvc.yaml` (stage `prepare`), `params.yaml`, `dvc.lock` (empreintes des données) ; aucune donnée ni aucun poids dans Git.
- **MLflow** : paramètres, courbes par epoch, métriques de test par classe, artefacts (poids, matrice de confusion, analyse par taille), Model Registry avec alias.
- **API FastAPI** : `/health`, `/model/metadata`, `/predict`, `/predict/visualize` ; schémas Pydantic, Swagger sur `/docs`, erreurs 400 et 503, modèle ONNX chargé une seule fois au démarrage, inférence hors de la boucle asynchrone.
- **Frontend React** : glisser-déposer, galerie d'exemples, webcam, canvas de boîtes colorées par classe, bandeau d'alerte, latences client et serveur.
- **Docker Compose** : services `mlflow`, `backend`, `frontend` (images multi-stage) lancés par `docker-compose up -build`.
- **GitHub Actions** : black, flake8, 18 tests pytest, eslint, build du frontend, build Docker.

Règle d'alerte. Une tête est conforme si le centre d'un casque se trouve dans sa boîte agrandie de 50 % vers le haut. On part de `head` (abondante) plutôt que de `person`.

7 Limites et perspectives

- Classes EPI rares (`helmet`, `safety-vest`) : fausses alertes possibles ; piste : sur-échantillonnage ciblé ou ajout d'un dataset de chantiers.
- Budget GPU limité (Kaggle) : 20 à 30 epochs seulement, loin d'un entraînement complet.
- Scènes surtout de bureaux : la démo webcam détecte bien `head`, `face`, `hands`, moins bien les casques sans accessoire réel.
- Perspectives : modèle `s`, résolution 1280, suivi multi-objets, quantification INT8.

Références

[1] H. M. Ahmad, A. Rahimi, *SH17: A Dataset for Human Safety and Personal Protective Equipment Detection in Manufacturing Industry*, arXiv:2407.04590, 2024.

[2] G. Jocher et al., *Ultralytics YOLO*, https://github.com/ultralytics/ultralytics.